

```
"""
```

hasher.py – GRUS Audit Pipeline Stage 9

Specification: GRUS-STD-001-v1.0 Section 3 Stage 9

Computes SHA-256 hashes of submission code, manifest, and dataset snapshots for inclusion in the certification record.

Published by: Green Recursive Utility Service LLC

```
"""
```

```
import os, hashlib
```

```
from typing import Dict
```

```
from dataclasses import dataclass
```

```
@dataclass
```

```
class HashingResult:
```

```
    passed: bool
```

```
    manifest_hash: str = ""
```

```
    verify_py_hash: str = ""
```

```
    constants_py_hash: str = ""
```

```
    failure_reason: str = ""
```

```
def sha256_file(path: str) -> str:
```

```
    if not os.path.exists(path): return ""
```

```
    h = hashlib.sha256()
```

```
    with open(path, "rb") as f:
```

```
        for chunk in iter(lambda: f.read(65536), b''):
```

```
            h.update(chunk)
```

```
    return h.hexdigest()
```

```
def compute_hashes(repo_path: str) -> HashingResult:
```

```
    m = sha256_file(os.path.join(repo_path, "GRUS_MANIFEST.yaml"))
```

```
    v = sha256_file(os.path.join(repo_path, "verify.py"))
```

```
    c = ""
```

```
    for candidate in ("svcf_constants.py", "constants.py", "grus_constants.py"):
```

```
        path = os.path.join(repo_path, candidate)
```

```
        if os.path.exists(path):
```

```
            c = sha256_file(path); break
```

```
    if not m: return HashingResult(passed=False, failure_reason="GRUS_MANIFEST.yaml hash failed")
```

```
    if not v: return HashingResult(passed=False, failure_reason="verify.py hash failed")
```

```
    if not c: return HashingResult(passed=False, failure_reason="constants module hash failed")
```

```
    return HashingResult(passed=True, manifest_hash=m, verify_py_hash=v, constants_py_hash=c)
```

```
if __name__ == "__main__":
```

```
    import sys
```

```
    if len(sys.argv) < 2:
```

```
        print("Usage: python hasher.py <repo_path>"); sys.exit(0)
```

```
    r = compute_hashes(sys.argv[1])
```

```
    if r.passed:
```

```
        print(f"[PASS] manifest={r.manifest_hash[:16]}... verify={r.verify_py_hash[:16]}...")
```

```
constants={r.constants_py_hash[:16]}...)
else:
    print(f"[FAIL] {r.failure_reason}"); sys.exit(1)
```